

Local RSS Intelligence System — Phased Build Plan

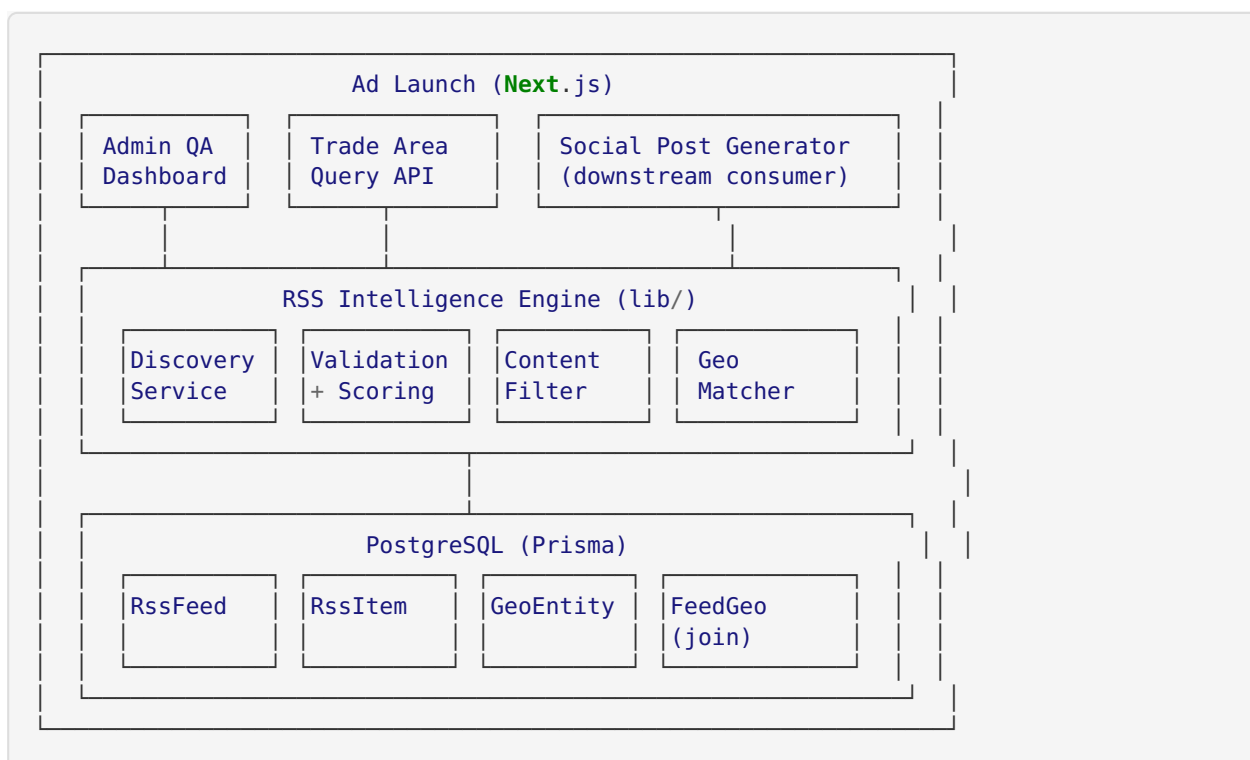
Purpose: Discover, validate, classify, geotag, and store RSS feeds so Ad Launch’s automation engine can pull only safe, relevant local content by trade area for automated business social posting.

Last Updated: April 7, 2026

Table of Contents

1. [Architecture Overview](#)
 2. [Phase 1: Requirements + Schema + Content Policy](#)
 3. [Phase 2: Geography Reference Layer](#)
 4. [Phase 3: Pilot Feed Discovery](#)
 5. [Phase 4: Feed Validation + Freshness Scoring + Dedupe](#)
 6. [Phase 5: Geographic Coverage Assignment](#)
 7. [Phase 6: Item-Level Content Filtering](#)
 8. [Phase 7: Trade Area Query Engine](#)
 9. [Phase 8: Admin QA Tools + Manual Overrides](#)
 10. [Phase 9: Export/API Layer](#)
 11. [Phase 10: Nationwide Rollout + Monitoring](#)
 12. [MVP Recommendation](#)
 13. [Pilot Rollout Recommendation](#)
 14. [Major Risks & Mitigations](#)
 15. [Example Queries](#)
-

Architecture Overview



Key Design Principles

- **PostgreSQL-native**, PostGIS-ready (geometry columns deferred, but lat/lng stored from Phase 2)
- **Many-to-many** geography coverage via explicit join table with confidence scores
- **Confirmed vs. Inferred** ZIP coverage tracked separately
- **Full audit trail** — every content decision (block/allow) logged with reason
- **Source quality hierarchy** — official/local > regional > national aggregator
- **Hard-block by default** on sexual/adult and political/opinion content

Phase 1: Requirements + Schema + Content Policy

Objective

Establish the data model, content policy rules, and classification taxonomy before writing any ingestion code.

Tasks

#	Task	Owner	Est.
1.1	Define RSS source taxonomy (local_news, gov_meeting, event, weather, school, community, chamber_of_commerce, police_blotter, sports_local, lifestyle)	Product	2h
1.2	Define content policy document — hard-block categories, soft-filter categories, allow-list overrides	Product	3h
1.3	Design Prisma schema for all new models (see below)	Eng	4h
1.4	Define feed quality scoring rubric (freshness, consistency, format compliance, content density)	Eng	2h
1.5	Document geographic hierarchy: State → County → City/Town → ZIP (many-to-many city↔ZIP)	Eng	2h
1.6	Define API contract for Trade Area Query (input: list of ZIPs/ city/county → output: ranked safe feed items)	Eng	3h

Data Model Changes

```
// === CONTENT POLICY ===

model ContentPolicy {
  id          String      @id @default(cuid())
  category    String      // e.g. "sexual_adult", "political_opinion", "violence_graphic"
  action      String      // "hard_block", "soft_filter", "allow"
  description  String?    @db.Text
  isActive    Boolean     @default(true)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  @@unique([category])
}

// === GEOGRAPHY ===

model GeoState {
  id          String      @id @default(cuid())
  code        String      @unique // "CO", "TX"
  name        String      // "Colorado", "Texas"
  fipsCode    String?     @unique // "08"
  counties    GeoCounty[]
  createdAt   DateTime    @default(now())
}

model GeoCounty {
  id          String      @id @default(cuid())
  name        String      // "El Paso County"
  fipsCode    String?     @unique // "08041"
  stateId     String
  state       GeoState    @relation(fields: [stateId], references: [id])
  cities      GeoCity[]
  createdAt   DateTime    @default(now())

  @@index([stateId])
}

model GeoCity {
  id          String      @id @default(cuid())
  name        String      // "Colorado Springs"
  countyId    String
  county      GeoCounty   @relation(fields: [countyId], references: [id])
  latitude    Float?      // PostGIS-ready
  longitude   Float?      // PostGIS-ready
  population  Int?        // For prioritization
  cityZips    GeoCityZip[]
  createdAt   DateTime    @default(now())

  @@index([countyId])
  @@index([name, countyId])
}

model GeoZip {
  id          String      @id @default(cuid())
  code        String      @unique // "80903"
  latitude    Float?
  longitude   Float?
  cityZips    GeoCityZip[]
  feedGeos    FeedGeo[]
  createdAt   DateTime    @default(now())
}
```

```

// Many-to-many: a ZIP can span multiple cities, a city has multiple ZIPs
model GeoCityZip {
  id          String    @id @default(cuid())
  cityId      String
  zipId       String
  isPrimary   Boolean   @default(false) // Is this the "main" city for this ZIP?
  city        GeoCity   @relation(fields: [cityId], references: [id])
  zip         GeoZip    @relation(fields: [zipId], references: [id])

  @@unique([cityId, zipId])
  @@index([zipId])
  @@index([cityId])
}

// === RSS FEEDS ===

model RssFeed {
  id          String    @id @default(cuid())
  url         String    @unique
  title       String?
  description  String?   @db.Text
  siteUrl     String?    // Homepage of the source
  sourceType  String    @default("unknown") // local_news, gov_meeting, event,
etc.
  sourceQuality String @default("unverified") // official, trusted, community,
aggregator, unverified
  status      String    @default("pending") // pending, active, stale, broken
, blocked, retired

  // Scoring
  freshnessScore Float? // 0-100, computed from publish frequency
  qualityScore   Float? // 0-100, composite of multiple signals
  lastFetchedAt  DateTime?
  lastItemDate   DateTime? // Most recent item publish date
  fetchErrorCount Int    @default(0)
  consecutiveErrors Int   @default(0)
  avgItemsPerWeek Float? // Publishing frequency

  // Deduplication
  contentHash  String? // Hash of feed metadata for dedupe
  canonicalUrl String? // Resolved canonical URL after redirects

  // Discovery metadata
  discoveredBy String? // "manual", "google_search", "sitemap_crawl",
"opml_import"
  discoveredAt  DateTime @default(now())
  verifiedBy    String?   // Admin user ID who verified
  verifiedAt    DateTime?

  // Audit
  notes        String?   @db.Text
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt

  items        RssItem[]
  feedGeos     FeedGeo[]
  feedAudits   FeedAudit[]

  @@index([status])
  @@index([sourceType])
  @@index([sourceQuality])
  @@index([freshnessScore])
}

```

```
// === FEED ↔ GEOGRAPHY (many-to-many with confidence) ===

model FeedGeo {
  id          String    @id @default(cuid())
  feedId      String
  zipId       String
  coverageType String    // "confirmed" or "inferred"
  confidence  Float      @default(0.5) // 0.0-1.0
  source      String?    // How coverage was determined: "manual", "domain_geo", "content_nlp", "about_page"
  feed        RssFeed    @relation(fields: [feedId], references: [id], onDelete: Cascade)
  zip         GeoZip     @relation(fields: [zipId], references: [id])
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  @@unique([feedId, zipId])
  @@index([zipId])
  @@index([feedId])
  @@index([coverageType])
  @@index([confidence])
}

// === RSS ITEMS ===

model RssItem {
  id          String    @id @default(cuid())
  feedId      String
  guid        String      // RSS item guid/link for dedupe
  title       String?
  description  String?    @db.Text
  link        String?
  pubDate     DateTime?
  author      String?

  // Content classification
  contentHash String?      // Hash for cross-feed dedupe
  categories  String[]    @default([]) // Tags from RSS + our classification

  // Filtering
  filterStatus String    @default("pending") // pending, approved, blocked, manual_review
  filterReason String?   @db.Text              // Why it was blocked/approved
  blockedCategory String? // Which policy category triggered the block

  // Quality
  relevanceScore Float? // 0-100, how relevant to local audience

  // Timestamps
  fetchedAt   DateTime   @default(now())
  createdAt   DateTime   @default(now())

  feed        RssFeed    @relation(fields: [feedId], references: [id], onDelete: Cascade)
  itemAudits  ItemAudit[]

  @@unique([feedId, guid])
  @@index([feedId])
  @@index([pubDate])
  @@index([filterStatus])
  @@index([contentHash])
}
```

```
// === AUDIT TRAIL ===

model FeedAudit {
  id          String   @id @default(cuid())
  feedId      String
  action      String   // "status_change", "geo_assigned", "geo_removed", "quality_recalc", "manual_override"
  oldValue    String?  @db.Text
  newValue    String?  @db.Text
  reason      String?  @db.Text
  performedBy String?   // User ID or "system"
  createdAt   DateTime @default(now())
  feed        RssFeed  @relation(fields: [feedId], references: [id], onDelete: Cascade)

  @@index([feedId])
  @@index([createdAt])
}

model ItemAudit {
  id          String   @id @default(cuid())
  itemId      String
  action      String   // "auto_blocked", "auto_approved", "manual_approved", "manual_blocked", "reclassified"
  category    String?  // Content policy category that triggered action
  confidence  Float?   // Model confidence for auto-decisions
  reason      String?  @db.Text
  performedBy String?   // User ID or "system"
  createdAt   DateTime @default(now())
  item        RssItem  @relation(fields: [itemId], references: [id], onDelete: Cascade)

  @@index([itemId])
  @@index([createdAt])
}
```

Services/Scripts Needed

- `lib/rss/content-policy.ts` — Policy rule engine (load from DB, evaluate content)
- `prisma/seed-content-policies.ts` — Seed default hard-block and soft-filter rules

QA Checks

- [] Schema passes `prisma validate` and `prisma db push (dev)`
- [] Content policy seed script populates all default categories
- [] API contract document reviewed and approved
- [] Source taxonomy covers ≥ 10 local source types

Exit Criteria

- ☒ Schema migrated to dev DB without data loss on existing tables
- ☒ Content policy document signed off
- ☒ API contract for trade area query finalized
- ☒ Taxonomy of 10+ source types documented

Phase 2: Geography Reference Layer

Objective

Populate the State → County → City → ZIP hierarchy so feeds can be geotagged against a canonical reference.

Tasks

#	Task	Owner	Est.
2.1	Source authoritative ZIP/city/county data-set (Census ZCTA, HUD crosswalk, or SimpleMaps free tier)	Eng	2h
2.2	Write <code>scripts/import-geo-data.ts</code> — parse CSV, populate GeoState/County/City/Zip/CityZip	Eng	6h
2.3	Handle edge cases: ZIP codes spanning county/state lines, independent cities, census-designated places	Eng	3h
2.4	Add lat/lng centroids to GeoZip and GeoCity (PostGIS-ready but stored as Float for now)	Eng	2h
2.5	Build lookup API: <code>GET /api/rss/geo/lookup?q=80903</code> → returns city, county, state chain	Eng	2h
2.6	Validate coverage: 50 states, 3,000+ counties, 30,000+ cities, 41,000+ ZIPs	Eng	2h

Data Model Changes

None beyond Phase 1 schema. This phase is data population only.

Services/Scripts Needed

- `scripts/import-geo-data.ts` — One-time import from CSV/JSON
- `lib/rss/geo-lookup.ts` — Fast ZIP ↔ city ↔ county ↔ state resolution
- `app/api/rss/geo/lookup/route.ts` — REST endpoint for geo queries

Data Sources (Recommended)

1. **Primary:** [SimpleMaps US ZIP Codes Database](https://simplemaps.com/data/us-zips) (https://simplemaps.com/data/us-zips) (free tier: ZIP, city, county, state, lat/lng, population)
2. **Supplemental:** Census Bureau ZCTA-to-County crosswalk for multi-county ZIPs
3. **FIPS codes:** Census Bureau FIPS lookup tables

QA Checks

- ☐ 50 states + DC loaded
- ☐ ≥3,100 counties loaded (3,143 actual)
- ☐ ≥41,000 ZIPs loaded
- ☐ Multi-city ZIPs have multiple GeoCityZip records
- ☐ Spot-check 10 known ZIPs across different states
- ☐ Lookup API returns correct hierarchy for edge cases (NYC boroughs, independent cities in VA)

Exit Criteria

- ☒ Full US geography loaded with <0.1% error rate on spot checks
- ☒ Lookup API responds in <50ms for single ZIP query
- ☒ City↔ZIP many-to-many relationships verified

Phase 3: Pilot Feed Discovery

Objective

Discover and catalog RSS feeds for 3–5 pilot states/metro areas to validate the discovery pipeline before nationwide rollout.

Tasks

#	Task	Owner	Est.
3.1	Select pilot regions: recommend CO (home state), TX (large), FL (diverse), NC (mid-size), and 1 rural state (e.g., MT)	Product	1h
3.2	Build <code>lib/rss/discovery.ts</code> — automated discovery via: (a) Google Search API (<code>site:*.rss</code> OR <code>inurl:feed</code>), (b) common RSS path probing (<code>/feed</code> , <code>/rss</code> , <code>/atom.xml</code>), (c) HTML <code><link rel="alternate" type="application/rss+xml"></code> extraction	Eng	8h
3.3	Build curated seed list of known high-quality local sources per pilot state (news-papers, TV stations, city gov sites, chambers of commerce)	Eng	4h
3.4	Write <code>scripts/discover-feeds.ts</code> — orchestrate discovery for a given state/region	Eng	3h
3.5	Classify discovered feeds by <code>sourceType</code> and <code>sourceQuality</code>	Eng	3h
3.6	Store all discovered feeds as <code>status: "pending"</code> in <code>RssFeed</code>	Eng	1h

Data Model Changes

None beyond Phase 1.

Services/Scripts Needed

- `lib/rss/discovery.ts` — Discovery engine (Google Search, path probing, HTML parsing)
- `lib/rss/source-classifier.ts` — Classify `sourceType` + `sourceQuality` using URL patterns and feed metadata
- `scripts/discover-feeds.ts` — CLI script for batch discovery
- `scripts/seed-pilot-feeds.ts` — Import curated seed list

Curated Source Priority

Tier 1 (official):	city/county .gov sites, school district feeds
Tier 2 (trusted):	local newspapers, local TV station news
Tier 3 (community):	local radio, community blogs, patch.com local
Tier 4 (aggregator):	Google News local, Apple News local ❌ DEPRIORITIZED

QA Checks

- [] ≥ 50 feeds discovered per pilot state
- [] $\geq 70\%$ of feeds have valid RSS/Atom XML
- [] Source classification accuracy $\geq 80\%$ on manual spot-check of 50 feeds
- [] No duplicate URLs (canonicalization working)
- [] Seed list covers top-10 population cities in each pilot state

Exit Criteria

- ☒ ≥ 250 feeds across 5 pilot states in `RssFeed` table
- ☒ Each feed classified by `sourceType` and `sourceQuality`
- ☒ Discovery pipeline repeatable for new states

Phase 4: Feed Validation + Freshness Scoring + Dedupe

Objective

Fetch every discovered feed, validate it parses correctly, score freshness/quality, and deduplicate equivalent feeds.

Tasks

#	Task	Owner	Est.
4.1	Build <code>lib/rss/fetcher.ts</code> — fetch + parse RSS/Atom with <code>rss-parser</code> or <code>fast-xml-parser</code>	Eng	4h
4.2	Build <code>lib/rss/scorer.ts</code> — compute <code>freshnessScore</code> (days since last item, avg frequency) and <code>qualityScore</code> (title presence, description length, image presence, consistent format)	Eng	4h
4.3	Build dedupe logic: canonical URL resolution (follow redirects, strip tracking params), content hash of feed title+siteUrl+first 3 item GUIDs	Eng	3h
4.4	Status transitions: pending → active (valid + fresh), pending → stale (>30 days old), pending → broken (parse error or 404)	Eng	2h
4.5	Write <code>scripts/validate-feeds.ts</code> — batch validate all pending feeds	Eng	2h
4.6	Set up scheduled task for ongoing re-validation (daily or triggered)	Eng	2h

Data Model Changes

None — all fields exist in Phase 1 schema.

Services/Scripts Needed

- `lib/rss/fetcher.ts` — Fetch + parse with timeout, retry, user-agent rotation
- `lib/rss/scorer.ts` — Freshness + quality scoring algorithms
- `lib/rss/deduper.ts` — Canonical URL resolution + content hash dedupe
- `scripts/validate-feeds.ts` — Batch validation CLI

Scoring Rubric

Freshness Score (0-100):

```
100: Published in last 24 hours
80:  Published in last 3 days
60:  Published in last 7 days
40:  Published in last 14 days
20:  Published in last 30 days
0:   No items in 30+ days → mark "stale"
```

Quality Score (0-100):

```
+25: All items have titles
+20: >80% items have descriptions >50 chars
+15: Feed has <link> to source site
+15: Consistent publish schedule (stddev < 3 days)
+10: Items have pubDate
+10: Items have author
+5:  Items have media/image
```

QA Checks

- ☐ Parser handles RSS 2.0, Atom 1.0, and RSS 0.91
- ☐ Freshness scores match manual verification on 20 feeds
- ☐ Dedupe catches ≥90% of equivalent feeds (same source, different URL formats)
- ☐ Broken feeds (404, parse error) correctly marked
- ☐ Stale feeds (>30d) correctly marked

Exit Criteria

- ☒ All pilot feeds validated with freshness + quality scores
- ☒ Active feeds have freshnessScore > 20
- ☒ <5% false-positive dedupe (incorrectly merging different feeds)
- ☒ Status distribution: ≥60% active, ≤20% stale, ≤20% broken

Phase 5: Geographic Coverage Assignment

Objective

Assign each feed to the ZIPs/cities/counties it covers, with confidence scores differentiating confirmed vs. inferred coverage.

Tasks

#	Task	Owner	Est.
5.1	Build <code>lib/rss/geo-tagger.ts</code> — multi-signal geographic assignment	Eng	8h
5.2	Signal: Domain-based geo (e.g., <code>gazette.com</code> → Colorado Springs, <code>denver-post.com</code> → Denver metro)	Eng	3h
5.3	Signal: About/contact page scraping for city/state/ZIP mentions	Eng	4h
5.4	Signal: Feed item content NLP — extract place names from recent items	Eng	4h
5.5	Signal: Manual override from admin (Phase 8)	Eng	1h
5.6	Confidence scoring: combine signals, output confirmed (≥ 0.8 from manual or 2+ agreeing signals) vs inferred (< 0.8)	Eng	3h
5.7	Write <code>scripts/geotag-feeds.ts</code> — batch assign geography to all active feeds	Eng	2h

Data Model Changes

None — FeedGeo join table defined in Phase 1.

Services/Scripts Needed

- `lib/rss/geo-tagger.ts` — Multi-signal geo assignment engine
- `lib/rss/domain-geo-lookup.ts` — Known domain → city/ZIP mappings (curated + heuristic)

- `lib/rss/place-extractor.ts` — Extract place names from text using regex + known city list
- `scripts/geotag-feeds.ts` — Batch geotagging CLI

Confidence Scoring Matrix

Signal	Confidence	Type
Manual admin assignment	1.0	confirmed
Domain ☐ known city mapping	0.85	confirmed (if ☐ 2 signals agree)
About page has address/ZIP	0.80	confirmed
Content NLP (☐ 5 items cite same city)	0.65	inferred
Content NLP (☐ 2 items cite same city)	0.45	inferred
State-level only	0.20	inferred

Final confidence = `max(individual signals)`
 OR average of top 2 if they agree on same geo
 coverageType = confidence ☐ 0.8 ☐ "confirmed" : "inferred"

QA Checks

- [] ≥80% of active feeds have at least 1 FeedGeo record
- [] Confirmed coverage matches manual verification on 30 feeds
- [] No feed assigned to >50 ZIPs without manual review
- [] State-level feeds correctly get all county ZIPs (with low confidence)

Exit Criteria

- ☒ All active pilot feeds have geographic coverage assigned
- ☒ ≥50% of assignments are “confirmed” (confidence ≥0.8)
- ☒ Audit trail in FeedAudit for all geo assignments

Phase 6: Item-Level Content Filtering

Objective

Filter every RSS item to block sexual/adult and political/opinion content — the two hard-block categories — plus soft-filter other sensitive categories.

Tasks

#	Task	Owner	Est.
6.1	Build <code>lib/rss/content-filter.ts</code> — multi-layer content classification	Eng	8h
6.2	Layer 1: Keyword/regex blocklist for sexual/adult terms (high-precision, zero tolerance)	Eng	3h
6.3	Layer 1b: Keyword/regex blocklist for political/opinion terms (candidate names, party terms, editorial indicators)	Eng	3h
6.4	Layer 2: LLM-based classification for ambiguous content (via existing Abacus AI API) — batch classify title+description	Eng	4h
6.5	Layer 3: Source-level preemptive blocking (e.g., entire feed is political commentary → block all items)	Eng	2h
6.6	Write <code>scripts/filter-items.ts</code> — batch filter all pending items	Eng	2h
6.7	Build audit logging: every block/approve decision logged to <code>ItemAudit</code> with category + confidence + reason	Eng	2h

Data Model Changes

None — ItemAudit and filter fields defined in Phase 1.

Services/Scripts Needed

- `lib/rss/content-filter.ts` — Main filter orchestrator
- `lib/rss/keyword-blocklists.ts` — Curated keyword lists for hard-block categories
- `lib/rss/llm-classifier.ts` — LLM-based content classification (batch mode)
- `scripts/filter-items.ts` — Batch filtering CLI

Content Policy: Hard-Block Categories

```
CATEGORY: sexual_adult
ACTION: hard_block
SIGNALS:
  - Explicit sexual terms in title/description
  - Adult industry domains
  - NSFW image indicators
FALSE POSITIVE HANDLING:
  - Medical/health articles with anatomical terms ☐ soft_filter (manual review)
  - Dating tips articles ☐ allow if no explicit content

CATEGORY: political_opinion
ACTION: hard_block
SIGNALS:
  - Candidate names + election/campaign context
  - Party names (Republican, Democrat, etc.) in opinion context
  - Editorial/opinion section indicators ("Op-Ed", "Editorial", "Opinion", "Letter
to Editor")
  - Partisan policy advocacy language
FALSE POSITIVE HANDLING:
  - City council meeting minutes ☐ allow (factual government)
  - Local ballot measure factual coverage ☐ allow
  - "Mayor announces..." (factual local gov news) ☐ allow
```

Classification Flow

```
Item arrives → Keyword scan (fast, ~1ms)
├─ Hard match on sexual/adult keywords → BLOCK immediately
├─ Hard match on political/opinion keywords → BLOCK immediately
├─ Soft match (ambiguous) → send to LLM classifier
└─ No match → check source-level policy
    ├─ Source is pre-blocked → BLOCK
    └─ Source is clean → APPROVE

LLM Classifier (batch, ~500ms per batch of 20):
Input: "Classify this RSS item. Is it: safe_local_content, political_opinion, sexu-
al_adult, violence_graphic, other_sensitive, or unclear?"
Output: { category, confidence, reasoning }
Decision: confidence > 0.7 → auto-decide, else → manual_review
```

QA Checks

- [] 0% false negatives on sexual/adult (test with 50 known-bad items)
- [] 0% false negatives on political/opinion (test with 50 known-bad items)
- [] <5% false positives on safe local content (test with 200 known-good items)
- [] Audit trail complete for every item decision

- [] LLM classifier batch throughput ≥ 100 items/minute

Exit Criteria

- ☒ All pilot feed items classified
 - ☒ Zero sexual/adult content in approved items (verified by manual audit of 200 approved items)
 - ☒ Zero political/opinion content in approved items (verified by manual audit of 200 approved items)
 - ☒ $< 5\%$ false positive rate (safe content incorrectly blocked)
 - ☒ Every decision has an ItemAudit record
-

Phase 7: Trade Area Query Engine

Objective

Build the core query engine that downstream social post generation will call: "Give me safe, fresh, relevant RSS items for these ZIPs/cities."

Tasks

#	Task	Owner	Est.
7.1	Build <code>lib/rss/trade-area-engine.ts</code> — core query logic	Eng	6h
7.2	Input normalization: accept ZIPs, city names, county names, or state codes → resolve to canonical ZIP set	Eng	3h
7.3	Feed selection: query FeedGeo for matching ZIPs, rank by (confidence × qualityScore × freshnessScore)	Eng	3h
7.4	Item selection: from matching feeds, select approved items within date window, dedupe by contentHash	Eng	3h
7.5	Relevance scoring: boost items mentioning the specific city/ZIP, penalize generic state-level items	Eng	3h
7.6	Build <code>GET /api/rss/trade-area</code> endpoint	Eng	2h
7.7	Add pagination, date range, source type, and content category filters	Eng	2h
7.8	Response caching layer (in-memory TTL or Redis-like)	Eng	2h

Data Model Changes

None.

Services/Scripts Needed

- `lib/rss/trade-area-engine.ts` — Core query engine
- `lib/rss/zip-resolver.ts` — Resolve any geo input to canonical ZIP set
- `app/api/rss/trade-area/route.ts` — REST endpoint

API Contract

```
// GET /api/rss/trade-area?
// zips=80903,80904&limit=20&days=7&sourceTypes=local_news,event

interface TradeAreaRequest {
  zips?: string[];           // Direct ZIP codes
  cities?: string[];         // City names (resolved to ZIPs)
  counties?: string[];        // County names (resolved to ZIPs)
  states?: string[];          // State codes (resolved to ZIPs) – use sparingly
  limit?: number;             // Max items to return (default 20)
  days?: number;              // Look back N days (default 7)
  sourceTypes?: string[];     // Filter by source type
  minConfidence?: number;     // Minimum geo confidence (default 0.3)
  excludeInferred?: boolean; // Only return confirmed coverage feeds
}

interface TradeAreaResponse {
  items: {
    id: string;
    title: string;
    description: string;
    link: string;
    pubDate: string;
    feedTitle: string;
    feedSourceType: string;
    feedSourceQuality: string;
    geoConfidence: number;
    relevanceScore: number;
    categories: string[];
  }[];
  meta: {
    totalItems: number;
    feedsMatched: number;
    zipsSearched: number;
    queryTimeMs: number;
  };
}
```

QA Checks

- [] Query for Colorado Springs ZIPs returns local news, not Denver-only feeds
- [] Items are properly ranked (local > regional > state-level)
- [] No blocked items appear in results
- [] Pagination works correctly
- [] Response time <500ms for typical trade area (5-20 ZIPs)

Exit Criteria

-  Trade area query returns relevant, safe items for all pilot regions

-  Response time <500ms for 95th percentile queries
-  Zero blocked content in query results (verified by automated test)
-  Confirmed-coverage feeds ranked higher than inferred

Phase 8: Admin QA Tools + Manual Overrides

Objective

Build an admin dashboard for feed QA, manual geo assignment, content review, and override capabilities.

Tasks

#	Task	Owner	Est.
8.1	Build admin page: / admin/rss — feed list with filters (status, sourceType, state, quality)	Eng	4h
8.2	Feed detail view: metadata, geo assignments, recent items, audit history	Eng	4h
8.3	Manual geo assignment: search for ZIP/city, assign with confidence=1.0, coverage-Type="confirmed"	Eng	3h
8.4	Manual content review queue: items in manual_review status, approve/block with reason	Eng	4h
8.5	Feed status overrides: force-activate, force-block, retire feeds	Eng	2h
8.6	Bulk operations: select multiple feeds → assign to geo, change status, re-score	Eng	3h
8.7	Dashboard stats: total feeds by status, items by filter status, coverage heatmap by state	Eng	3h
8.8	Export audit log as CSV	Eng	1h

Data Model Changes

```
// Add admin role to User model (or separate AdminUser)
model User {
  // ... existing fields ...
  role String @default("user") // "user", "admin"
}
```

Services/Scripts Needed

- `app/admin/rss/page.tsx` — Feed management dashboard
- `app/admin/rss/[id]/page.tsx` — Feed detail + edit page
- `app/admin/rss/review/page.tsx` — Content review queue
- `app/api/admin/rss/*` — CRUD endpoints for feeds, geo, items
- `lib/rss/admin-service.ts` — Admin business logic

QA Checks

- [] Admin can view, filter, and sort all feeds
- [] Manual geo assignment creates FeedGeo with confidence=1.0 and FeedAudit record
- [] Content review approve/block updates ItemAudit
- [] Non-admin users cannot access admin pages
- [] Audit trail is complete for all manual actions

Exit Criteria

- ☒ Admin can manage full feed lifecycle from dashboard
- ☒ All manual actions create audit records
- ☒ Content review queue processes items in <3 clicks per item
- ☒ Dashboard loads in <2s with 1,000+ feeds

Phase 9: Export/API Layer

Objective

Build the integration layer that downstream social post generation systems consume.

Tasks

#	Task	Owner	Est.
9.1	Formalize trade area API for production use (auth, rate limiting, versioning)	Eng	3h
9.2	Build POST /api/rss/curate — given a business profile (URL, ZIPs, industry), return curated content suggestions with social post angles	Eng	6h
9.3	Build content summarization: use LLM to create social-post-ready summaries from RSS items	Eng	4h
9.4	Build GET /api/rss/feed-health — monitoring endpoint for feed/coverage stats	Eng	2h
9.5	Connect to existing Ad Launch pipeline: when generating social posts, pull from RSS intelligence	Eng	4h
9.6	API documentation (OpenAPI/Swagger or markdown)	Eng	2h

Data Model Changes

```
// Track curated content selections for analytics
model CuratedSelection {
  id          String @id @default(cuid())
  analysisId  String? // Link to Ad Launch analysis
  itemId      String // RSS item selected
  tradeAreaZips String[] // ZIPs used in query
  usedInPost  Boolean @default(false)
  createdAt   DateTime @default(now())

  @@index([analysisId])
  @@index([itemId])
}
```

Services/Scripts Needed

- `app/api/rss/curate/route.ts` — Content curation endpoint
- `lib/rss/content-summarizer.ts` — LLM-based summarization for social posts
- `app/api/rss/feed-health/route.ts` — Monitoring endpoint
- `docs/rss-api.md` — API documentation

QA Checks

- [] Curate endpoint returns 5-10 relevant items per business within 3 seconds
- [] Summaries are social-post-ready (appropriate length, tone, no blocked content)
- [] Feed health endpoint accurately reports coverage gaps
- [] Rate limiting prevents abuse

Exit Criteria

- ☒ Downstream social post generator successfully consumes curated content
- ☒ API documentation complete
- ☒ End-to-end test: business URL → RSS content → social post draft

Phase 10: Nationwide Rollout + Monitoring

Objective

Expand from pilot states to full US coverage, with automated monitoring and alerting.

Tasks

#	Task	Owner	Est.
10.1	Run discovery pipeline for all 50 states + DC	Eng	8h
10.2	Batch validate + score all discovered feeds	Eng	4h
10.3	Batch geotag all active feeds	Eng	4h
10.4	Set up recurring scheduled task: daily feed refresh + item ingestion	Eng	3h
10.5	Set up recurring scheduled task: weekly freshness re-scoring	Eng	2h
10.6	Build monitoring dashboard: coverage gaps by state, stale feed alerts, content filter stats	Eng	4h
10.7	Alerting: notify admin when a state has <10 active feeds or >20% stale rate	Eng	2h
10.8	Performance optimization: add DB indexes for common query patterns, consider materialized views	Eng	4h
10.9	Load testing: simulate 10K trade area queries/hour	Eng	3h

Data Model Changes

```
// Coverage tracking for monitoring
model GeoCoverage {
  id String @id @default(cuid())
  stateCode String
  totalZips Int
  coveredZips Int
  confirmedZips Int
  activeFeeds Int
  staleFeeds Int
  avgFreshness Float?
  avgQuality Float?
  computedAt DateTime @default(now())

  @@index([stateCode])
  @@index([computedAt])
}
```

Services/Scripts Needed

- `scripts/nationwide-discovery.ts` — Full US discovery orchestrator
- `lib/rss/monitoring.ts` — Coverage gap detection + alerting
- `app/admin/rss/coverage/page.tsx` — Coverage heatmap dashboard
- Scheduled tasks: daily feed refresh, weekly re-scoring, daily coverage stats

QA Checks

- [] ≥ 45 states have ≥ 20 active feeds
- [] Total active feeds $\geq 2,000$ nationwide
- [] $< 10\%$ stale rate across all feeds
- [] Trade area queries work for every state
- [] System handles 10K queries/hour without degradation

Exit Criteria

- ☒ 50-state coverage with monitoring
- ☒ Automated daily refresh running
- ☒ Admin alerted on coverage gaps
- ☒ System handles production load

MVP Recommendation

Scope: Phases 1-7 (core engine without admin UI)

Estimated timeline: 6–8 weeks with 1 engineer

MVP delivers:

- Full geography reference layer (US-wide)
- Feed discovery + validation for pilot states
- Content filtering with hard-block on sexual/adult + political/opinion
- Trade area query API ready for downstream consumption
- Audit trail for all content decisions

MVP defers:

- Admin dashboard (use direct DB queries + scripts for QA)
- Nationwide coverage (start with 5 states)
- Curated content summaries (return raw items)
- Production monitoring/alerting

Recommended MVP milestone: Demo trade area query for Colorado Springs returning 10+ safe, relevant, recent local news items.

Phase Grouping for MVP Sprints

Sprint 1 (Week 1-2): Phase 1 + Phase 2 – Schema + Geography
 Sprint 2 (Week 3-4): Phase 3 + Phase 4 – Discovery + Validation
 Sprint 3 (Week 5-6): Phase 5 + Phase 6 – Geo Assignment + Content Filtering
 Sprint 4 (Week 7-8): Phase 7 – Trade Area Query + Integration

Pilot Rollout Recommendation**Pilot Regions (5 states, diverse mix)**

State	Why	Target Cities	Expected Feed Count
CO	Home state, deep knowledge	Colorado Springs, Denver, Boulder, Fort Collins	80-120
TX	Largest state, urban+rural mix	Austin, Dallas, Houston, San Antonio, El Paso	100-150
FL	Tourism + diverse economy	Miami, Orlando, Tampa, Jacksonville	80-120
NC	Mid-size metros + rural	Charlotte, Raleigh, Asheville, Wilmington	60-90
MT	Rural stress test	Billings, Missoula, Bozeman, Great Falls	20-40

Pilot Success Metrics

- ≥ 300 total active feeds across pilot states
- $\geq 60\%$ confirmed geo coverage
- 0% blocked content in approved items (hard requirement)
- $< 5\%$ false positive rate on content filtering
- Trade area query returns ≥ 5 items for any city $> 50K$ population in pilot states
- Trade area query returns ≥ 1 item for any city $> 10K$ population

Pilot → Nationwide Gate

Proceed to Phase 10 only when:

1. Pilot success metrics met
 2. Admin has reviewed ≥ 100 items with $< 2\%$ override rate
 3. Content filter false positive rate $< 3\%$
 4. No sexual/adult or political content leaked to approved items in 2+ weeks
-

Major Risks & Mitigations

Risk 1: RSS Availability — Many local sources don't have RSS feeds

- **Likelihood:** HIGH
- **Impact:** LOW coverage for small/rural markets
- **Mitigation:**
 - Probe multiple common paths (/feed , /rss , /atom.xml , /?feed=rss2)
 - Check for JSON Feed format (/feed.json)
 - For major sources without RSS, consider building lightweight page-change monitors (Phase 10+)
 - Prioritize Patch.com local, which has consistent RSS across many small towns

Risk 2: Content Filter False Positives — Blocking safe local content

- **Likelihood:** MEDIUM
- **Impact:** Reduced useful content for social posts
- **Mitigation:**
 - Start with high-precision keyword lists (fewer, more specific terms)
 - Use LLM classifier only for ambiguous cases
 - Manual review queue catches false positives early
 - Track false positive rate weekly; tune thresholds iteratively
 - Whitelist known safe sources (e.g., weather, school closings)

Risk 3: Content Filter False Negatives — Letting bad content through

- **Likelihood:** LOW (for sexual/adult), MEDIUM (for political)
- **Impact:** CRITICAL — brand safety violation for businesses
- **Mitigation:**
 - Multi-layer approach: keyword + LLM + source-level blocking
 - “Political” is harder to define than “sexual” — maintain evolving keyword list
 - During election seasons, increase LLM classifier aggressiveness
 - Monthly audit of 500 random approved items
 - Kill-switch: block entire feed instantly if any item leaks

Risk 4: Geographic Accuracy — Assigning feeds to wrong locations

- **Likelihood:** MEDIUM
- **Impact:** Irrelevant content served to businesses
- **Mitigation:**
 - Multi-signal approach (domain, about page, content NLP)
 - Separate confirmed vs. inferred — downstream can filter by confidence

- Admin override for important feeds
- Regular spot-checking of geo assignments

Risk 5: Feed Rot — Feeds go stale or disappear

- **Likelihood:** HIGH (10-20% annual churn)
- **Impact:** Degraded coverage over time
- **Mitigation:**
 - Daily automated freshness checks
 - Status transitions: active → stale → broken → retired
 - Weekly re-discovery for states with declining feed counts
 - Admin alerts when a region drops below coverage threshold

Risk 6: Database Scale — Millions of items over time

- **Likelihood:** HIGH at nationwide scale
- **Impact:** Slow queries, storage costs
- **Mitigation:**
 - Partition RssItem by month (or archive items > 90 days)
 - Aggressive indexing on query-hot columns (pubDate, filterStatus, feedId)
 - Consider materialized views for trade area query
 - BRIN indexes on timestamp columns for range queries
 - Set item retention policy (keep 90 days hot, archive older)

Example Queries

“Safe feeds in customer trade area” — Colorado Springs business at ZIPs 80903, 80904, 80905

```
-- Query 1: Find all active feeds covering these ZIPs with confirmed or high-
confidence inferred coverage
SELECT DISTINCT
  f.id,
  f.title,
  f.url,
  f."sourceType",
  f."sourceQuality",
  f."freshnessScore",
  f."qualityScore",
  fg."coverageType",
  fg.confidence,
  gz.code AS zip_code
FROM "RssFeed" f
JOIN "FeedGeo" fg ON fg."feedId" = f.id
JOIN "GeoZip" gz ON gz.id = fg."zipId"
WHERE gz.code IN ('80903', '80904', '80905')
  AND f.status = 'active'
  AND fg.confidence >= 0.3
ORDER BY
  fg."coverageType" ASC, -- confirmed first
  fg.confidence DESC,
  f."qualityScore" DESC;
```

```
-- Query 2: Get recent safe items from those feeds (last 7 days)
SELECT
  i.id,
  i.title,
  i.description,
  i.link,
  i."pubDate",
  i."relevanceScore",
  i.categories,
  f.title AS feed_title,
  f."sourceType",
  f."sourceQuality",
  fg.confidence AS geo_confidence,
  fg."coverageType"
FROM "RssItem" i
JOIN "RssFeed" f ON f.id = i."feedId"
JOIN "FeedGeo" fg ON fg."feedId" = f.id
JOIN "GeoZip" gz ON gz.id = fg."zipId"
WHERE gz.code IN ('80903', '80904', '80905')
  AND f.status = 'active'
  AND i."filterStatus" = 'approved'
  AND i."pubDate" >= NOW() - INTERVAL '7 days'
  AND fg.confidence >= 0.3
ORDER BY
  fg."coverageType" ASC,
  (fg.confidence * COALESCE(f."qualityScore", 50) * COALESCE(i."relevanceScore", 50))
DESC,
  i."pubDate" DESC
LIMIT 20;
```

```
-- Query 3: Coverage report – how many ZIPs in El Paso County, CO have feed coverage?
SELECT
  gz.code AS zip_code,
  gc.name AS city_name,
  COUNT(DISTINCT fg."feedId") AS active_feeds,
  COUNT(DISTINCT CASE WHEN fg."coverageType" = 'confirmed' THEN fg."feedId" END) AS confirmed_feeds,
  MAX(fg.confidence) AS max_confidence
FROM "GeoZip" gz
JOIN "GeoCityZip" gcz ON gcz."zipId" = gz.id
JOIN "GeoCity" gc ON gc.id = gcz."cityId"
JOIN "GeoCounty" gco ON gco.id = gc."countyId"
LEFT JOIN "FeedGeo" fg ON fg."zipId" = gz.id
  AND EXISTS (SELECT 1 FROM "RssFeed" f WHERE f.id = fg."feedId" AND f.status = 'active')
WHERE gco.name = 'El Paso County'
  AND gco."stateId" = (SELECT id FROM "GeoState" WHERE code = 'CO')
GROUP BY gz.code, gc.name
ORDER BY active_feeds DESC;
```

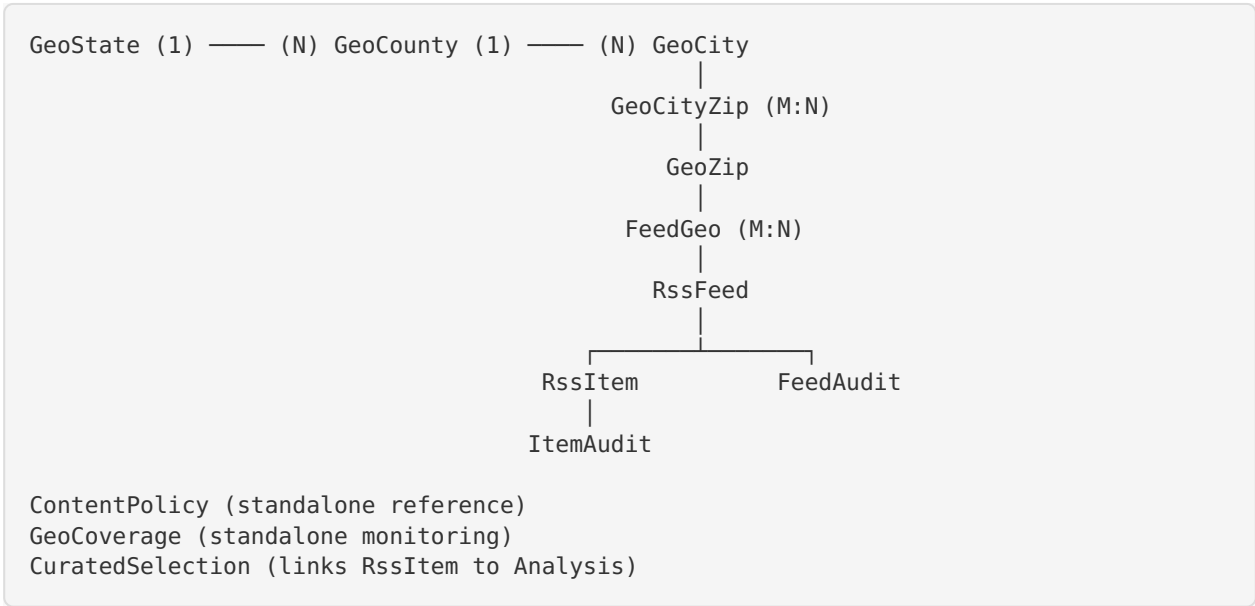


```
-- Query 4: Audit trail – show all content filtering decisions for a specific feed in
the last 24h
SELECT
  ia.action,
  ia.category,
  ia.confidence,
  ia.reason,
  ia."performedBy",
  ia."createdAt",
  i.title AS item_title,
  i."filterStatus"
FROM "ItemAudit" ia
JOIN "RssItem" i ON i.id = ia."itemId"
WHERE i."feedId" = 'feed_abc123'
  AND ia."createdAt" >= NOW() - INTERVAL '24 hours'
ORDER BY ia."createdAt" DESC;
```

Prisma (TypeScript) Equivalents

```
// Trade area query – safe items for ZIPs
const items = await prisma.rssItem.findMany({
  where: {
    filterStatus: 'approved',
    pubDate: { gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000) },
    feed: {
      status: 'active',
      feedGeos: {
        some: {
          zip: { code: { in: ['80903', '80904', '80905'] } },
          confidence: { gte: 0.3 },
        },
      },
    },
  },
  include: {
    feed: {
      select: {
        title: true,
        sourceType: true,
        sourceQuality: true,
        feedGeos: {
          where: {
            zip: { code: { in: ['80903', '80904', '80905'] } },
          },
          select: { coverageType: true, confidence: true },
        },
      },
    },
  },
  orderBy: [{ pubDate: 'desc' }],
  take: 20,
});
```

Appendix A: Complete Model Relationship Diagram



Appendix B: Technology Stack

Component	Technology	Notes
Database	PostgreSQL (existing)	PostGIS extension deferred but lat/lng stored
ORM	Prisma	Existing in project
RSS Parsing	rss-parser or fast-xml-parser	yarn add
Content Classification	Abacus AI LLM API (existing)	gpt-4.1-mini for batch classification
Geo Data	SimpleMaps + Census ZCTA	Free tier sufficient
Scheduling	Abacus AI Daemon Tasks	Existing infrastructure
Admin UI	Next.js pages	Within existing app
Monitoring	Custom dashboard + email alerts	Via existing notification system

Appendix C: Estimated Timeline

Phase	Duration	Dependencies
Phase 1: Schema + Policy	1 week	None
Phase 2: Geography	1 week	Phase 1
Phase 3: Discovery	1.5 weeks	Phase 2
Phase 4: Validation	1 week	Phase 3
Phase 5: Geo Assignment	1.5 weeks	Phase 2 + 4
Phase 6: Content Filtering	1.5 weeks	Phase 1 + 4
Phase 7: Query Engine	1.5 weeks	Phase 5 + 6
MVP Complete	~8 weeks	
Phase 8: Admin Tools	2 weeks	Phase 7
Phase 9: Export/API	1.5 weeks	Phase 7
Phase 10: Nationwide	2 weeks	Phase 8 + 9
Full System	~14 weeks	